Table des matières

Labo 06 — Modéliser les autres faits 2
Avant de commencer 3
Étape 1 — Les quatre sources de l’après-vente 3
Étape 2 — Trois sources, trois comportements 4
Étape 3 — Le staging des véhicules et des clients 5
Étape 4 — Le staging des enregistrements et des réclamations 6
Étape 5 — Déplier l’historique en jalons 8
Étape 6 — La dimension du véhicule 10
Étape 7 — La dimension du client, et ce qu’elle ne porte pas 11
Étape 8 — Le fait des enregistrements 11
Étape 9 — Le fait d’accumulation 12
Étape 10 — Les tests de la nouvelle étoile 14
Étape 11 — Le témoin de l’après-vente 17
Étape 12 — Ce que l’étoile répond 18
Étape 13 — Ce que l’usine livre, et ce qu’on en prend 19
Étape 14 — Le registre des actifs, et la dimension à deux niveaux 21
Étape 15 — Le pivot des étiquettes 22
Étape 16 — L’instantané périodique 24
Étape 17 — Le fait sans mesure 25
Étape 18 — Les tests, et le témoin de l’usine 26
Étape 19 — L’incident, d’un bout à l’autre 28
Étape 20 — La matrice de bus, remplie 30
Récapitulatif 31
Dépannage 32

© 2026 iAlexMG. Tous droits réservés.

Labo 06 — Modéliser les autres faits

Labo · Cours 6/9 — Formation Analytics Engineering

Ce que vous allez faire. Bâtir les deux étoiles qui manquent à Boréal. Celle de l'après-vente d'abord, avec les ventes au détail et les réclamations de garantie : une source qui renvoie l'objet complet à chaque changement de statut, un historique JSON à déplier en jalons, et votre premier fait d'accumulation, dont la ligne se met à jour au lieu de s'ajouter. Celle de l'usine ensuite, avec un instantané périodique heure par heure et un fait sans mesure. Objectif de fin : `dbt build` monte vingt-sept modèles et vérifie soixante-dix-huit choses en quatre secondes, et les cinq faits racontent ensemble un incident que personne n'avait relié.

Deux terminaux servent dans ce labo. L'Anaconda Prompt, environnement `formations` actif, sert à dbt et à Git ; il est ouvert dans `Projet_Boreal_intermediaire/02_ae`. La ligne de commande DuckDB sert à vérifier ce que dbt a écrit, et elle doit être fermée quand dbt travaille.

La liste de contrôle du cours :

- ☐ Les quatre sources de l'après-vente déclarées
- ☐ Les trois comportements de source constatés sur les données
- ☐ `stg_vehicules` et `stg_clients` : 27 775 et 23 527 lignes
- ☐ `stg_enregistrements` et `stg_reclamations` : 26 732 et 3 348 lignes
- ☐ Les jalons dépliés : 9 447 étapes pour 3 348 réclamations
- ☐ `dim_vehicule` : 27 775 véhicules, et `dim_client` sans donnée personnelle
- ☐ `fct_enregistrements` : 26 732 lignes, sur la version d'époque du concessionnaire
- ☐ `fct_reclamations` : 3 348 lignes, quatre jalons, trois durées
- ☐ Les 35 tests de la nouvelle étoile, au vert
- ☐ Le témoin : 329 réclamations hors garantie, 329 refus au bon motif
- ☐ L'inspection renforcée, chiffrée
- ☐ Le forage transversal par la dimension du véhicule
- ☐ Les quatre sources de l'usine déclarées, dont le schéma `argent`
- ☐ `dim_actif` : 23 actifs, 6 zones et 17 équipements

© 2026 iAlexMG. Tous droits réservés.

- ☐ `fct_disponibilite_actifs` : 22 090 lignes, sans trou
- ☐ `fct_production` : 27 775 véhicules, sans mesure
- ☐ Le témoin de l'usine : 1 833 véhicules des deux côtés
- ☐ L'incident lu dans quatre faits
- ☐ La matrice de bus vérifiée dans le graphe

Le labo se fait en deux séances. La première va jusqu'à l'étape 12, la seconde porte l'usine.

Avant de commencer

Ce labo part du projet tel que le Cours 05 l'a laissé. Vérifiez-le dans l'Anaconda Prompt, depuis `Projet_Boreal_intermediaire/02_ae` :

```
dbt build --exclude snp_concessionnaire
```

`dbt build` doit rendre `PASS=35 WARN=0 ERROR=0` sur 36 nœuds. Fermez ensuite la ligne de commande DuckDB si elle est ouverte, par `.exit`.

Étape 1 — Les quatre sources de l'après-vente

Théorie : `Theorie_06_AE.md`, section 1.

L'après-vente vit dans quatre tables de la `RAW`, et une cinquième vient de l'ERP. Ajoutez-les à `models/staging/_sources.yml`, sous les cinq déjà déclarées :

```
      - name: erp_zserie
        description: "Un véhicule assemblé par ligne, sa commande et son expédition."
      - name: garanties_clients
        description: "Les clients, dans leur version de fin de mois."
      - name: garanties_enregistrements
        description: "Une ligne par véhicule vendu au détail et enregistré."
      - name: garanties_reclamations
        description: "Les réclamations de garantie, un objet JSON par changement de statut."
```

La règle du Cours 03 tient toujours : on ne déclare que ce qu'un modèle lit. Les visites d'entretien et les consentements restent hors du projet tant qu'aucun modèle ne les lit.

```
dbt ls --resource-type source
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : neuf sources.

Étape 2 — Trois sources, trois comportements

Théorie : `Theorie_06_AE.md`, section 3.

Avant d'écrire une ligne de staging, regardez comment chaque source se comporte. Dans DuckDB :

```
select 'enregistrements' as source, count(*) as lignes,
       count(distinct no_enregistrement) as objets
from raw.garanties_enregistrements
union all
select 'clients', count(*), count(distinct client_id)
from raw.garanties_clients
union all
select 'reclamations', count(*), count(distinct objet ->> 'reclamation_id')
from raw.garanties_reclamations;
```

Source	Lignes	Objets	Ce que ça veut dire
enregistrements	26 732	26 732	une ligne par objet : rien ne revient
clients	25 379	23 527	la fiche revient quand elle change
reclamations	5 623	3 348	l'objet revient à chaque changement de statut

Trois comportements dans un même dossier livré par un même système. Regardez une réclamation qui a traversé trois extraits :

```
select _lot, objet ->> 'statut' as statut, objet -> 'historique' as historique
from raw.garanties_reclamations
where objet ->> 'reclamation_id' = 'RC0000040'
order by _lot;
```

Lot	Statut	Historique
2022-05	SOUMISE	une étape
2022-06	APPROUVEE	deux étapes
2022-07	PAYEE	trois étapes

© 2026 iAlexMG. Tous droits réservés.

L'historique est cumulatif : la dernière version porte tout. C'est elle qu'on garde, et c'est la raison pour laquelle l'ordre des deux gestes compte.

Étape 3 — Le staging des véhicules et des clients

Théorie : `Theorie_06_AE.md`, section 6.

Dans `models/staging/stg_vehicules.sql` :

```sql
with courant as (

    select
        *,
        row_number() over (
            partition by sernr
            order by _lot desc
        ) as rang

    from {{ source('raw', 'erp_zserie') }}

)

select
    sernr as numero_serie,
    matnr as code_article,
    werks as division,
    arbpl as ligne_assemblage,
    try_strptime(erdat, '%Y%m%d')::date as date_assemblage,
    try_strptime(wadat_ist, '%Y%m%d')::date as date_expedition,
    wadat_ist <> '00000000' as est_expedie,
    vbeln as numero_commande,
    posnr as numero_poste,
    _lot as lot_source

from courant
where rang = 1
```

Rien de neuf : les quatre gestes du Cours 03. La date d'expédition vaut `00000000` tant que le véhicule n'est pas sorti de l'usine, et `try_strptime` en fait une valeur vide plutôt qu'une erreur.

Puis `models/staging/stg_clients.sql`, et regardez surtout ce qu'il ne prend pas :

© 2026 iAlexMG. Tous droits réservés.

```sql
with courant as (

    select
        *,
        row_number() over (
            partition by client_id
            order by _lot desc
        ) as rang

    from {{ source('raw', 'garanties_clients') }}

)

select
    client_id as code_client,
    ville,
    province,
    langue,
    statut,
    statut = 'EFFACE' as est_efface,
    date_creation::date as date_creation,
    _lot as lot_source

from courant
where rang = 1
```

La source porte le prénom, le nom, le courriel, le téléphone, l'adresse et le code postal. Aucun ne monte. La règle est celle de la théorie : une colonne de données personnelles ne monte que si une analyse devient impossible sans elle.

```
dbt run --select stg_vehicules stg_clients
```

```sql
select
    (select count(*) from staging.stg_vehicules) as vehicules,
    (select count(*) from staging.stg_vehicules where est_expedie) as expedies,
    (select count(*) from staging.stg_clients) as clients,
    (select count(*) from staging.stg_clients where est_efface) as effaces;
```

Attendu : 27 775 véhicules, dont 27 698 expédiés ; 23 527 clients, dont 1 effacé.

Étape 4 — Le staging des enregistrements et des réclamations

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_AE.md`, section 3.

`models/staging/stg_enregistrements.sql` est le plus simple du projet, et c'est normal : la source ne renvoie jamais deux fois le même enregistrement, donc aucune déduplication.

```
select
    no_enregistrement as numero_enregistrement,
    sernr as numero_serie,
    client_id as code_client,
    concessionnaire as code_concessionnaire,
    date_vente::date as date_vente,
    date_enregistrement::date as date_enregistrement,
    annee_modele::integer as annee_modele,
    politique_garantie,
    debut_garantie::date as debut_garantie,
    fin_garantie::date as fin_garantie,
    type_inspection,
    module_telematique = 'OUI' as est_connecte,
    _lot as lot_source

from {{ source('raw', 'garanties_enregistrements') }}
```

`models/staging/stg_reclamations.sql` est l'inverse : la déduplication d'abord, puis un objet JSON à aplatir champ par champ. L'historique, lui, reste en JSON : il se déplie à l'étape suivante.

```
with courant as (

    select
        objet,
        _lot,
        row_number() over (
            partition by objet ->> 'reclamation_id'
            order by _lot desc
        ) as rang

    from {{ source('raw', 'garanties_reclamations') }}

)

select
    objet ->> 'reclamation_id' as numero_reclamation,
    objet ->> 'sernr' as numero_serie,
    objet ->> 'client_id' as code_client,
```


© 2026 iAlexMG. Tous droits réservés.

```
    objet ->> 'concessionnaire' as code_concessionnaire,
    (objet ->> 'date_ouverture')::date as date_ouverture,
    objet ->> 'cause' as cause,
    objet ->> 'statut' as statut,
    objet ->> 'code_anomalie' as code_anomalie,
    objet ->> 'motif_refus' as motif_refus,
    (objet -> 'main_oeuvre' ->> 'heures')::decimal(18, 2) as heures_main_oeuvre,
    (objet -> 'main_oeuvre' ->> 'taux_horaire')::decimal(18, 2) as taux_horaire,
    (objet ->> 'montant_total')::decimal(18, 2) as montant_total,
    objet -> 'historique' as historique,
    _lot as lot_source

from courant
where rang = 1
```

Deux opérateurs se croisent ici et ils ne font pas la même chose. `->` rend du JSON, ce qu'on veut pour `historique` et pour descendre dans `main_oeuvre` ; `->>` rend du texte, ce qu'on veut pour tout ce qui sera une colonne. Les parenthèses autour d'un `->>` avant un `::` ne sont pas décoratives : sans elles, DuckDB lit l'expression dans le mauvais ordre.

```
dbt run --select stg_enregistrements stg_reclamations
```

```
select
    (select count(*) from staging.stg_enregistrements) as enregistrements,
    (select count(*) from staging.stg_reclamations) as reclamations,
    (select count(*) from staging.stg_reclamations where motif_refus is not null) as avec_motif;
```

Attendu : 26 732, 3 348, et 395 réclamations refusées portant un motif.

Étape 5 — Déplier l'historique en jalons

Théorie : `Theorie_06_AE.md`, section 3.

Le fait veut une colonne par jalon ; la source livre une liste. Le passage de l'un à l'autre se range dans la couche `INTERMEDIATE`, pour que le modèle du fait reste lisible.

Dans `models/intermediate/int_jalons_reclamations.sql` :

```
with etapes as (

    select
        r.numero_reclamation,
```


© 2026 iAlexMG. Tous droits réservés.

```
        etape.statut,
        etape.date

    from {{ ref('stg_reclamations') }} as r,
        lateral (
            select unnest(
                json_transform(r.historique, '[{"statut": "VARCHAR", "date": "DATE"}]')
            )
        ) as h (etape)

)

select
    numero_reclamation,
    max(case when statut = 'SOUMISE' then date end) as date_soumission,
    max(case when statut = 'APPROUVEE' then date end) as date_approbation,
    max(case when statut = 'REFUSEE' then date end) as date_refus,
    max(case when statut = 'PAYEE' then date end) as date_paiement,
    count(*) as etapes_franchies

from etapes
group by 1
```

Trois mécanismes se lisent dans ces quinze lignes. `json_transform` donne un type à chaque champ, ce qui évite de recaster ensuite. `unnest` sur une liste JSON rend une ligne par élément, et le `lateral` permet de le faire pour chaque réclamation. Et le `max(case when …)` est le pivot : il replie ces lignes en une seule, avec une colonne par statut.

Renvoi : `lateral` est au Cours 07 de SQL, section 7 ; aplatir et typer du JSON, à son Cours 08, sections 5 et 6 ; le pivot par agrégat conditionnel, à son Cours 06, sections 6 et 7.

Le modèle est `ephemeral`, comme celui du Cours 04 : il n'existe que dans le SQL du fait.

```
dbt run --select int_jalons_reclamations+ --exclude fct_reclamations
```

Rien ne se construit, et c'est attendu : un modèle `ephemeral` ne crée aucun objet. Pour le vérifier, lancez plutôt le comptage à la main dans DuckDB :

```
select count(*) as etapes, count(distinct r.numero_reclamation) as reclamations
from staging.stg_reclamations r,
     lateral (select unnest(
         json_transform(r.historique, '[{"statut": "VARCHAR", "date": "DATE"}]')
     )) as h (etape);
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : 9 447 étapes pour 3 348 réclamations. Faites le même comptage sur `raw.garanties_reclamations`, sans déduplication : 13 455 étapes. Ce sont les 4 008 étapes des versions intermédiaires, qui donneraient un compte d'étapes faux avec des dates justes.

Étape 6 — La dimension du véhicule

Théorie : `Theorie_06_AE.md`, section 4.

Première des deux dimensions conformes du cours. Elle servira à l'étoile de l'après-vente, à celle de l'usine, et à la télémétrie du Cours 07.

Dans `models/marts/dim_vehicule.sql` :

```
select
    {{ dbt_utils.generate_surrogate_key(['v.numero_serie']) }} as cle_vehicule,
    v.numero_serie,
    v.code_article,
    a.gamme,
    a.annee_modele,
    v.ligne_assemblage,
    v.date_assemblage,
    v.date_expedition,
    v.est_expedie

from {{ ref('stg_vehicules') }} as v
inner join {{ ref('stg_articles') }} as a
    on a.code_article = v.code_article
```

La gamme et l'année-modèle viennent de l'article, et elles sont donc écrites à deux endroits, ici et dans `dim_produit`. C'est une dénormalisation, elle est voulue, et le Cours 04 l'a déjà dite : une dimension a le droit d'être dénormalisée quand cela évite une jointure à tous ceux qui la liront.

```
dbt run --select dim_vehicule
```

```
select count(*) as vehicules, count(distinct cle_vehicule) as cles,
       count(*) filter (where gamme is null) as sans_gamme
from marts.dim_vehicule;
select gamme, count(*) as n from marts.dim_vehicule group by 1 order by 1;
```

Attendu : 27 775 véhicules, 27 775 clés, 0 sans gamme. Aucune pièce n'y figure, puisqu'une pièce n'a pas de numéro de série.

© 2026 iAlexMG. Tous droits réservés.

Étape 7 — La dimension du client, et ce qu'elle ne porte pas

Théorie : `Theorie_06_AE.md`, section 6.

Dans `models/marts/dim_client.sql` :

```
select
    {{ dbt_utils.generate_surrogate_key(['c.code_client']) }} as cle_client,
    c.code_client,
    c.ville,
    c.province,
    r.nom_province,
    c.langue,
    c.est_efface,
    c.date_creation

from {{ ref('stg_clients') }} as c
left join {{ ref('stg_regions') }} as r
    on r.code_province = c.province
```

```
dbt run --select dim_client
```

```
describe marts.dim_client;
```

Huit colonnes, et pas une de plus : la clé, le code du client, la ville, la province et son libellé, la langue, le drapeau d'effacement et la date de création. Un tableau de bord qui voudrait écrire au client passera par le système source, avec les droits qui vont avec.

Regardez la fiche effacée, celle du client qui a exercé son droit le 16 septembre 2025 :

```
select code_client, ville, province, langue, est_efface, date_creation
from marts.dim_client
where est_efface;
```

Une ligne, `CL0011163`, de Rouyn-Noranda. Ses données personnelles ont disparu de la source ; ce que l'entrepôt gardait n'avait pas à disparaître, parce qu'il n'y en avait pas.

Étape 8 — Le fait des enregistrements

Théorie : `Theorie_06_AE.md`, section 5.

Dans `models/marts/fct_enregistrements.sql` :

© 2026 iAlexMG. Tous droits réservés.

```
select
    e.date_vente as cle_date_vente,
    d.cle_concessionnaire,
    {{ dbt_utils.generate_surrogate_key(['e.numero_serie']) }} as cle_vehicule,
    {{ dbt_utils.generate_surrogate_key(['e.code_client']) }} as cle_client,
    {{ dbt_utils.generate_surrogate_key(['v.code_article']) }} as cle_produit,
    e.numero_enregistrement,
    e.politique_garantie,
    e.type_inspection,
    e.est_connecte,
    e.debut_garantie,
    e.fin_garantie,
    datediff('day', e.date_vente, e.date_enregistrement) as jours_avant_enregistrement,
    datediff('month', e.debut_garantie, e.fin_garantie + 1) as mois_de_couverture

from {{ ref('stg_enregistrements') }} as e
inner join {{ ref('stg_vehicules') }} as v
    on v.numero_serie = e.numero_serie
inner join {{ ref('dim_concessionnaire') }} as d
    on d.code_concessionnaire = e.code_concessionnaire
   and e.date_vente >= d.valide_de
   and (d.valide_jusqua is null or e.date_vente < d.valide_jusqua)
```

Trois clés se recalculent par hachage, celle du concessionnaire se joint. C'est la règle du Cours 05 : une dimension de type 2 ne se recalcule pas, puisque son code ne désigne plus une ligne. La date qui choisit la version est celle de la vente au détail, et non celle de l'enregistrement, qui peut suivre de trois jours.

```
dbt run --select fct_enregistrements
```

```
select count(*) as lignes, count(distinct numero_enregistrement) as objets,
       count(*) filter (where cle_concessionnaire is null) as sans_concessionnaire
from marts.fct_enregistrements;
select politique_garantie, count(*) as n, min(mois_de_couverture) as mois
from marts.fct_enregistrements group by 1 order by 1;
```

Attendu : 26 732 lignes pour autant d'enregistrements, aucune sans concessionnaire ; puis GAR-2022 sur 17 309 ventes avec 12 mois de couverture, GAR-2024 sur 9 423 avec 24 mois. La politique révisée du 1er août 2024 se lit dans les données sans qu'on ait eu à l'écrire nulle part.

Étape 9 — Le fait d'accumulation

Théorie : `Theorie_06_AE.md`, section 2.

© 2026 iAlexMG. Tous droits réservés.

Dans `models/marts/fct_reclamations.sql` :

```
select
    r.date_ouverture as cle_date_ouverture,
    j.date_approbation as cle_date_approbation,
    j.date_refus as cle_date_refus,
    j.date_paiement as cle_date_paiement,
    d.cle_concessionnaire,
    {{ dbt_utils.generate_surrogate_key(['r.numero_serie']) }} as cle_vehicule,
    {{ dbt_utils.generate_surrogate_key(['r.code_client']) }} as cle_client,
    r.numero_reclamation,
    r.cause,
    r.statut,
    r.code_anomalie,
    r.motif_refus,
    r.date_ouverture > e.fin_garantie as est_hors_garantie,
    datediff('day', j.date_soumission, coalesce(j.date_approbation, j.date_refus))
        as jours_jusqua_la_decision,
    datediff('day', j.date_approbation, j.date_paiement) as jours_jusquau_paiement,
    datediff('day', j.date_soumission, j.date_paiement) as jours_jusqua_la_cloture,
    r.heures_main_oeuvre,
    r.heures_main_oeuvre * r.taux_horaire as montant_main_oeuvre,
    r.montant_total - r.heures_main_oeuvre * r.taux_horaire as montant_pieces,
    r.montant_total

from {{ ref('stg_reclamations') }} as r
inner join {{ ref('int_jalons_reclamations') }} as j
    on j.numero_reclamation = r.numero_reclamation
inner join {{ ref('stg_enregistrements') }} as e
    on e.numero_serie = r.numero_serie
inner join {{ ref('dim_concessionnaire') }} as d
    on d.code_concessionnaire = r.code_concessionnaire
   and r.date_ouverture >= d.valide_de
   and (d.valide_jusqua is null or r.date_ouverture < d.valide_jusqua)
```

Quatre colonnes de dates, dont trois peuvent être vides : ce sont les jalons. Trois durées, qui sont vides tant que leur jalon d'arrivée ne l'est pas. Et une jointure de plus, vers l'enregistrement, qui apporte la fin de garantie et permet de dire si la réclamation est arrivée dans les délais.

Le montant des pièces se calcule par soustraction plutôt qu'en additionnant la liste des pièces du JSON, et ce n'est pas de la paresse : le montant total de la source est arrondi au cent, et recalculer les composantes séparément donne 1,69 $ d'écart sur 1 459 608,53 $. Le fait garde la mesure que la source porte, et en déduit le reste, pour que ses trois montants soient cohérents entre eux.

```
dbt run --select fct_reclamations
```


© 2026 iAlexMG. Tous droits réservés.

```sql
select
    count(*) as reclamations,
    count(cle_date_ouverture) as ouvertes,
    count(cle_date_approbation) as approuvees,
    count(cle_date_refus) as refusees,
    count(cle_date_paiement) as payees,
    round(avg(jours_jusqua_la_decision), 2) as jours_decision,
    round(avg(jours_jusquau_paiement), 2) as jours_paiement,
    round(avg(jours_jusqua_la_cloture), 2) as jours_cloture
from marts.fct_reclamations;
```

Mesure	Attendu
Réclamations	3 348
Ouvertes	3 348
Approuvées	2 905
Refusées	395
Payées	2 799
Jours jusqu’à la décision	5,27
Jours jusqu’au paiement	18,88
Jours jusqu’à la clôture	24,16

Les 443 réclamations qui n’ont ni approbation ni refus sont les dossiers en cours à la date de l’extrait. Regardez maintenant la ligne unique de la réclamation qui avait traversé trois extraits à l’étape 2 :

```sql
select numero_reclamation, cle_date_ouverture, cle_date_approbation, cle_date_paiement,
       statut, jours_jusqua_la_decision, jours_jusquau_paiement, jours_jusqua_la_cloture
from marts.fct_reclamations
where numero_reclamation = 'RC0000040';
```

Une ligne, trois jalons remplis : ouverte le 2022-05-30, approuvée le 2022-06-07, payée le 2022-07-07, soit 8 jours de décision, 30 de paiement et 38 en tout. Les trois versions livrées par la source sont devenues une ligne qui raconte le dossier.

Étape 10 — Les tests de la nouvelle étoile

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_AE.md`, sections 2 et 4.

Le Cours 05 a posé les règles ; ici on les applique sans les rediscuter. Ajoutez à `models/marts/_marts.yml`, avant le bloc `unit_tests:`, les tests des deux dimensions et des deux faits. Pour `dim_vehicule` :

```yaml
  - name: dim_vehicule
    description: "Une ligne par véhicule assemblé, identifié par son numéro de série."
    columns:
      - name: cle_vehicule
        data_tests:
          - unique
          - not_null
      - name: numero_serie
        data_tests:
          - unique
          - not_null
      - name: gamme
        data_tests:
          - not_null
          - accepted_values:
              arguments:
                values: ['Motoneige', 'Motomarine', 'Hors-route']
```

Pour `fct_reclamations`, les tests portent sur le grain, les clés et les deux colonnes codées :

```yaml
  - name: fct_reclamations
    description: >
      Le fait d'accumulation des réclamations de garantie, au grain de la réclamation. Sa
      ligne se met à jour à chaque jalon franchi, au lieu de s'ajouter.
    columns:
      - name: numero_reclamation
        data_tests:
          - unique
          - not_null
      - name: cle_date_ouverture
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_date')
                field: date_jour
      - name: cle_date_paiement
        data_tests:
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
      - relationships:
          arguments:
            to: ref('dim_date')
            field: date_jour
  - name: cle_vehicule
    data_tests:
      - not_null
      - relationships:
          arguments:
            to: ref('dim_vehicule')
            field: cle_vehicule
  - name: statut
    data_tests:
      - not_null
      - accepted_values:
          arguments:
            values: ['SOUMISE', 'APPROUVEE', 'REFUSEE', 'PAYEE']
  - name: cause
    data_tests:
      - not_null
      - accepted_values:
          arguments:
            values: ['REFROIDISSEMENT', 'MOTEUR', 'ELECTRIQUE', 'TRANSMISSION', 'CHASSIS', 'LIVRAISON']
```

Remarquez la clé du paiement : elle porte un `relationships` sans `not_null`, et c'est voulu. Un jalon vide est un dossier en cours, pas une anomalie ; le test de relation, qui ignore les valeurs absentes, est exactement le bon outil ici. C'est le seul endroit du projet où l'absence de `not_null` à côté est une décision et non un oubli.

Écrivez de même les tests de `dim_client` et de `fct_enregistrements`, sur le même moule : clé unique et non vide, relation vers chaque dimension, valeurs acceptées sur les colonnes codées.

Un fait d'accumulation appelle en plus un test que les autres n'ont pas : l'ordre de ses jalons. Dans `tests/assert_jalons_dans_l_ordre.sql` :

```sql
with r as (

    select
        numero_reclamation,
        cle_date_ouverture,
        cle_date_approbation,
        cle_date_refus,
        cle_date_paiement,
        statut
```


© 2026 iAlexMG. Tous droits réservés.

```
    from {{ ref('fct_reclamations') }}

)

select *
from r
where cle_date_approbation < cle_date_ouverture
   or cle_date_refus < cle_date_ouverture
   or cle_date_paiement < cle_date_approbation
   or (cle_date_paiement is not null and cle_date_approbation is null)
   or (cle_date_refus is not null and cle_date_approbation is not null)
```

Les trois premières conditions vérifient la chronologie, les deux dernières la logique du processus : on ne paie pas un dossier qu'on n'a pas approuvé, et on ne refuse pas un dossier approuvé.

```
dbt build --exclude snp_concessionnaire
```

Attendu : `PASS=78 WARN=0 ERROR=0` sur 79 nœuds, en trois secondes. Le projet compte maintenant 21 modèles et 58 tests de données, dont 35 arrivés avec ce cours.

Étape 11 — Le témoin de l'après-vente

Théorie : `Theorie_06_AE.md`, section 7.

Le fait des ventes se contrôlait contre un total que l'ERP calculait de son côté. Ici, personne ne tient de total indépendant. Le témoin est donc une règle de métier recalculée : une réclamation ouverte après la fin de la garantie doit être refusée pour ce motif.

Dans `tests/assert_refus_hors_garantie.sql` :

```
with r as (

    select
        numero_reclamation,
        est_hors_garantie,
        statut,
        motif_refus

    from {{ ref('fct_reclamations') }}

)
```


© 2026 iAlexMG. Tous droits réservés.

```sql
select *
from r
where est_hors_garantie != (motif_refus = 'Hors de la période de garantie')
   or (est_hors_garantie and statut != 'REFUSEE')
```

```
dbt test --select assert_refus_hors_garantie
```

Attendu : au vert. Puis, dans DuckDB, regardez ce que le test garantit :

```sql
select
    count(*) filter (where est_hors_garantie) as hors_garantie,
    count(*) filter (where motif_refus = 'Hors de la période de garantie') as motif_hors_garantie,
    count(*) filter (where statut = 'REFUSEE') as refusees
from marts.fct_reclamations;
```

Attendu : 329, 329 et 395. Votre calcul de la couverture et la décision du portail des garanties tombent sur les mêmes 329 dossiers. Les 66 refus qui restent portent un autre motif, l'usage non conforme au manuel, que rien dans les données ne permet de recalculer : un témoin n'a pas à tout couvrir, il a à être indépendant.

Étape 12 — Ce que l'étoile répond

Théorie : `Theorie_06_AE.md`, sections 4 et 5.

Deux questions pour finir, et ce sont elles qui justifient le travail.

La première vient du directeur de la qualité. Depuis mars 2025, 75 concessionnaires font une inspection avant livraison renforcée, et personne n'a encore dit si cela change quelque chose. La réponse demande les deux faits et la dimension qu'ils partagent :

```sql
with ventes as (
    select cle_vehicule, type_inspection
    from marts.fct_enregistrements
    where cle_date_vente between date '2025-03-01' and date '2025-12-31'
),
livraison as (
    select distinct cle_vehicule
    from marts.fct_reclamations
    where cause = 'LIVRAISON'
)
select
    v.type_inspection,
    count(*) as ventes,
```


© 2026 iAlexMG. Tous droits réservés.

```
    count(l.cle_vehicule) as avec_reclamation_livraison,
    round(100.0 * count(l.cle_vehicule) / count(*), 1) as taux
from ventes v
left join livraison l on l.cle_vehicule = v.cle_vehicule
group by 1
order by 1;
```

Inspection	Ventes	Avec réclamation de livraison	Taux
RENFORCEE	3 061	51	1,7 %
STANDARD	3 301	100	3,0 %

L'essai tient : les véhicules inspectés plus sévèrement reviennent presque deux fois moins pour un défaut de livraison. Une réserve à dire en même temps que le chiffre : les réclamations des ventes de fin 2025 arrivent en 2026, que l'entrepôt n'a pas encore ; le sens de l'écart est solide, sa valeur exacte bougera.

La seconde question est celle du forage transversal. Quel est le taux de réclamation par gamme, alors qu'aucun des deux faits ne porte la gamme ?

```
select
    d.gamme,
    count(distinct e.cle_vehicule) as vendus,
    count(distinct r.cle_vehicule) as avec_reclamation,
    round(100.0 * count(distinct r.cle_vehicule) / count(distinct e.cle_vehicule), 1) as taux
from marts.fct_enregistrements e
inner join marts.dim_vehicule d on d.cle_vehicule = e.cle_vehicule
left join marts.fct_reclamations r on r.cle_vehicule = e.cle_vehicule
group by 1
order by 4 desc;
```

Gamme	Vendus	Avec réclamation	Taux
Hors-route	9 305	1 141	12,3 %
Motomarine	6 488	779	12,0 %
Motoneige	10 939	1 212	11,1 %

Deux processus qui ne se connaissent pas, reliés par une dimension qu'ils partagent. C'est tout ce que « conforme » veut dire, et c'est ce que la matrice de bus du Cours 02 promettait.

Étape 13 — Ce que l'usine livre, et ce qu'on en prend

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_AE.md`, section 8.

Ici commence la seconde séance.

L'usine est instrumentée : un historien du SCADA, le système de supervision, contrôle et acquisition de données de l'usine, enregistre 58 points de mesure ; un registre décrit les actifs ; et une GMAO, la gestion de la maintenance assistée par ordinateur, tient les bons de travail. Rien de tout cela n'arrive ici sous sa forme brute. Data Engineering livre au Cours 08 trois tables de séries régulières, dans un schéma `argent`, et c'est ce que l'étoile lit.

Ce que l'usine produit	**Ce qu'Analytics Engineering en voit**
les points bruts de l'historien, à cadence variable	rien : ils restent au bronze
le registre des actifs et des étiquettes	`raw.usine_actifs`, `raw.usine_etiquettes`
la série horaire des 55 mesures et états	`argent.usine_horaire`
le compteur de chaque convoyeur	`argent.production_horaire`

Cette frontière est la même que celle du Cours 01, et elle se relit de la même façon : ce qui demande de connaître la bande morte d'un capteur, les codes de qualité OPC, la norme d'échange des automates industriels, et les valeurs de départ appartient à Data Engineering ; ce qui demande de connaître le métier appartient à ici.

Déclarez les quatre sources dans `models/staging/_sources.yml` :

```
      - name: usine_actifs
        description: "Le registre des actifs de l'usine : six zones et dix-sept équipements."
      - name: usine_etiquettes
        description: "Les 58 points mesurés de l'historien, rattachés à leur actif."

  - name: argent
    description: >
      Les séries de l'usine, rendues régulières à l'heure par Data Engineering au Cours 08.
      Les points bruts de l'historien restent au bronze et ne se lisent pas ici.
    schema: argent
    tables:
      - name: usine_horaire
        description: "Une étiquette et une heure : la valeur moyenne pondérée par le temps, et sa couverture."
      - name: production_horaire
        description: "Un compteur de convoyeur et une heure : la production tirée du cumul."
```

Les deux premières entrent sous la source `raw` déjà déclarée ; les deux dernières ouvrent une seconde source, `argent`, parce qu'elles vivent dans un autre schéma.

© 2026 iAlexMG. Tous droits réservés.

Regardez ce que ces tables portent, dans DuckDB :

```
select count(*) as lignes, count(distinct etiquette) as etiquettes,
       count(distinct heure) as heures, min(heure) as debut, max(heure) as fin
from argent.usine_horaire;
```

Attendu : 121 495 lignes, 55 étiquettes, 2 209 heures, du 1er octobre au 31 décembre 2025. Les 2 209 heures ne sont pas 92 fois 24 : la nuit du retour à l'heure normale en compte 25, et c'est la première conséquence d'une série horodatée en instants absolus.

Étape 14 — Le registre des actifs, et la dimension à deux niveaux

Théorie : `Theorie_06_AE.md`, section 10.

Deux vues de staging, sans surprise. Dans `models/staging/stg_actifs.sql` :

```
select
    actif_id,
    type_actif,
    parent_id as zone_id,
    nom,
    poste_travail_erp,
    mise_en_service::date as mise_en_service,
    fabricant,
    modele,
    puissance_nominale_kw::decimal(18, 2) as puissance_nominale_kw,
    parent_id is null as est_une_zone,
    _lot as lot_source

from {{ source('raw', 'usine_actifs') }}
```

Puis `models/staging/stg_etiquettes.sql` :

```
select
    etiquette,
    actif_id,
    grandeur,
    unite,
    bande_morte::decimal(18, 3) as bande_morte,
    plage_min::decimal(18, 3) as plage_min,
    plage_max::decimal(18, 3) as plage_max,
    description,
```


© 2026 iAlexMG. Tous droits réservés.

```
    _lot as lot_source

from {{ source('raw', 'usine_etiquettes') }}
```

Aucune déduplication : ces deux exports sont livrés en entier, une seule fois, et leur `_lot` vaut `complet`. Une vue de staging n'invente pas un traitement dont la source n'a pas besoin.

La dimension, dans `models/marts/dim_actif.sql`, porte les deux niveaux et résout la hiérarchie d'un seul côté :

```
select
    {{ dbt_utils.generate_surrogate_key(['a.actif_id']) }} as cle_actif,
    a.actif_id,
    a.type_actif,
    a.nom,
    a.est_une_zone,
    a.zone_id,
    z.nom as nom_zone,
    coalesce(a.poste_travail_erp, z.poste_travail_erp) as poste_travail_erp,
    a.fabricant,
    a.modele,
    a.puissance_nominale_kw,
    a.mise_en_service

from {{ ref('stg_actifs') }} as a
left join {{ ref('stg_actifs') }} as z
    on z.actif_id = a.zone_id
```

Le modèle se joint à lui-même : c'est ainsi qu'un équipement reçoit le nom de sa zone. Le `coalesce` sur le poste de travail fait descendre le code de l'ERP de la zone vers ses équipements, ce qui permettra de recouper l'usine et l'assemblage sans jointure supplémentaire.

```
dbt run --select stg_actifs stg_etiquettes dim_actif
```

```
select count(*) as actifs,
       count(*) filter (where est_une_zone) as zones,
       count(*) filter (where not est_une_zone) as equipements,
       count(*) filter (where poste_travail_erp is not null) as relies_a_l_erp
from marts.dim_actif;
```

Attendu : 23 actifs, 6 zones, 17 équipements, et 11 lignes reliées à l'ERP, les trois zones de lignes d'assemblage et les huit équipements qu'elles contiennent.

Étape 15 — Le pivot des étiquettes

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_AE.md`, section 8.

La série livrée porte une ligne par étiquette et par heure. L'instantané veut une ligne par actif et par heure, avec ses grandeurs en colonnes. C'est encore un pivot, comme celui des jalons, et il se range au même endroit.

Dans `models/intermediate/int_usine_par_actif.sql` :

```
with mesures as (

    select
        e.actif_id,
        e.grandeur,
        h.heure,
        h.valeur,
        h.couverture_s,
        h.points_douteux

    from {{ source('argent', 'usine_horaire') }} as h
    inner join {{ ref('stg_etiquettes') }} as e
        on e.etiquette = h.etiquette

)

select
    actif_id,
    heure,
    max(valeur) filter (where grandeur = 'MARCHE') as part_en_marche,
    max(valeur) filter (where grandeur = 'PUISSANCE') as puissance_moyenne_kw,
    max(valeur) filter (where grandeur = 'VIBRATION') as vibration_moyenne,
    max(couverture_s) filter (where grandeur = 'MARCHE') as couverture_s,
    sum(points_douteux) as points_douteux

from mesures
where actif_id in (
    select actif_id from {{ ref('stg_etiquettes') }} where grandeur = 'MARCHE'
)
group by 1, 2
```

Le filtre final mérite une phrase. Dix actifs sur dix-sept portent un état de marche, et ce sont eux qui ont une disponibilité : un réseau d'air ou un sécheur n'a pas d'état de marche, et lui en inventer un serait une mesure fabriquée. L'instantané porte donc dix actifs, pas dix-sept, et c'est une décision à documenter plutôt qu'à masquer.

© 2026 iAlexMG. Tous droits réservés.

Le nom de l'étiquette dit déjà l'actif et la grandeur, `UTL.CMP2.VIBRATION` par exemple. On ne le découpe pas : on passe par `stg_etiquettes`, qui porte le rattachement. Un nom composé est une convention, pas une clé étrangère, et il change le jour où l'intégrateur en décide autrement.

Étape 16 — L'instantané périodique

Théorie : `Theorie_06_AE.md`, section 8.

Dans `models/marts/fct_disponibilite_actifs.sql` :

```
select
    u.heure,
    (u.heure at time zone 'America/Toronto')::date as cle_date,
    {{ dbt_utils.generate_surrogate_key(['u.actif_id']) }} as cle_actif,
    u.part_en_marche,
    u.couverture_s,
    u.puissance_moyenne_kw,
    u.vibration_moyenne,
    u.points_douteux,
    u.couverture_s = 0 or u.part_en_marche is null as est_sans_mesure

from {{ ref('int_usine_par_actif') }} as u
```

Douze lignes, et trois décisions dedans. L'heure reste dans le fait, en plus de la clé de date : c'est le grain, et le perdre reviendrait à ne plus pouvoir distinguer deux heures du même jour. `est_sans_mesure` nomme ce qu'une valeur vide veut dire, pour qu'un rapport n'ait pas à le deviner.

Et la clé de date se calcule par une conversion écrite, `at time zone 'America/Toronto'`, jamais par un `heure::date` qui suivrait le fuseau de la session. Les deux donnent le même résultat sur un poste réglé à l'heure du Québec, et le second donnerait autre chose ailleurs : mesuré, 4 280 des 22 090 heures changent de jour entre l'heure locale et l'heure UTC, presque une sur cinq. Un modèle dont le résultat dépend du réglage de la machine qui le lance n'est pas un modèle.

Renvoi : l'heure de mur, l'instant et `at time zone` sont au Cours 02 de SQL, section 6 ; la journée vue d'un fuseau, et celles de 23 et 25 heures, à son Cours 09, section 1.

```
dbt run --select fct_disponibilite_actifs
```

```
select count(*) as lignes,
       count(distinct cle_actif) as actifs,
       count(distinct heure) as heures,
       count(*) filter (where couverture_s = 3600) as heures_pleines,
       count(*) filter (where couverture_s between 1 and 3599) as partielles,
       count(*) filter (where est_sans_mesure) as sans_mesure
from marts.fct_disponibilite_actifs;
```


© 2026 iAlexMG. Tous droits réservés.

Mesure	Attendu
Lignes	22 090
Actifs	10
Heures	2 209
Heures pleines	21 952
Heures partielles	81
Heures sans mesure	57

Dix actifs par 2 209 heures font exactement 22 090 lignes, sans trou : c'est la promesse d'un instantané. Regardez maintenant ce qu'il sait dire, et qu'un fait d'événements ne saurait pas :

```
select a.nom,
       round(sum(f.part_en_marche), 1) as heures_de_marche,
       round(100.0 * avg(f.part_en_marche), 1) as taux_d_utilisation,
       round(avg(f.puissance_moyenne_kw), 1) as puissance_moyenne_kw
from marts.fct_disponibilite_actifs f
inner join marts.dim_actif a on a.cle_actif = f.cle_actif
where a.type_actif = 'CONVOYEUR'
group by 1 order by 2 desc;
```

Les trois convoyeurs, sur les 2 209 heures du trimestre : 497,4 heures de marche pour celui des motoneiges, 237,4 pour celui des véhicules hors-route, 25,8 pour celui des motomarines. Le dernier chiffre n'est pas une anomalie, c'est la saison : on assemble les motoneiges à l'automne.

Additionner `part_en_marche` donne des heures, et c'est juste. Moyenner `puissance_moyenne_kw` donne une puissance, et c'est juste aussi parce que toutes les heures pèsent pareil. Additionner `puissance_moyenne_kw` ne donnerait rien du tout, et c'est ce que veut dire semi-additive.

Étape 17 — Le fait sans mesure

Théorie : `Theorie_06_AE.md`, sections 9 et 10.

Dans `models/marts/fct_production.sql` :

```
select
    v.date_assemblage as cle_date_assemblage,
    v.date_expedition as cle_date_expedition,
```


© 2026 iAlexMG. Tous droits réservés.

```
    {{ dbt_utils.generate_surrogate_key(['v.numero_serie']) }} as cle_vehicule,
    {{ dbt_utils.generate_surrogate_key(['v.code_article']) }} as cle_produit,
    {{ dbt_utils.generate_surrogate_key(['z.actif_id']) }} as cle_actif,
    v.numero_serie,
    v.numero_commande,
    v.numero_poste,
    v.est_expedie,
    datediff('day', v.date_assemblage, v.date_expedition) as jours_avant_expedition

from {{ ref('stg_vehicules') }} as v
inner join {{ ref('stg_actifs') }} as z
    on z.poste_travail_erp = v.ligne_assemblage
```

Quatre clés, deux dimensions dégénérées, un drapeau, et une seule chose qui ressemble à une mesure, le délai avant expédition. Aucune quantité : c'est un fait sans mesure, et son compte est son information.

La jointure vers `stg_actifs` est celle qui traduit `ASM-MN` en `LMN`. Elle se fait sur le staging et non sur la dimension, parce qu'on cherche la zone et non une de ses lignes : la dimension porterait les deux niveaux et il faudrait filtrer.

```
dbt run --select fct_production
```

```
select count(*) as lignes, count(distinct numero_serie) as vehicules,
       min(cle_date_assemblage) as premier, max(cle_date_assemblage) as dernier,
       count(*) filter (where est_expedie) as expedies,
       round(avg(jours_avant_expedition), 1) as jours_moyens
from marts.fct_production;
```

Attendu : 27 775 lignes pour autant de véhicules, du 17 janvier 2022 au 23 décembre 2025, 27 698 expédiés, 4,5 jours en moyenne entre l'assemblage et l'expédition.

Les deux faits de l'usine ne couvrent pas la même période, et il faut le savoir avant de les recouper : la production remonte à 2022, l'instantané commence le 1er octobre 2025, parce que l'historien ne livre que les 92 derniers jours. Sur cette fenêtre commune, 1 833 véhicules ont été assemblés.

Étape 18 — Les tests, et le témoin de l'usine

Théorie : `Theorie_06_AE.md`, section 11.

Les tests des trois nouveaux modèles suivent le moule des précédents : clé unique et non vide, relation vers chaque dimension, grain du fait. Celui de l'instantané est le seul qui change de forme, parce que son grain est un couple :

© 2026 iAlexMG. Tous droits réservés.

```yaml
  - name: fct_disponibilite_actifs
    description: >
      L'instantané périodique de l'usine, au grain d'un actif et d'une heure. Une ligne par
      heure existe même quand l'actif n'a rien produit, et c'est ce qui permet de compter les
      arrêts.
    data_tests:
      - dbt_utils.unique_combination_of_columns:
          arguments:
            combination_of_columns:
              - cle_actif
              - heure
```

Reste le témoin, et c'est le meilleur du cours. L'automate du convoyeur compte les véhicules terminés ; l'ERP enregistre des numéros de série. Personne n'a écrit ces deux compteurs ensemble, et ils doivent pourtant dire la même chose.

Dans `tests/assert_production_contre_compteur.sql` :

```sql
with compteur as (

    select
        case etiquette
            when 'LMN.CNV.COMPTEUR' then 'LMN'
            when 'LMM.CNV.COMPTEUR' then 'LMM'
            when 'LHR.CNV.COMPTEUR' then 'LHR'
        end as actif_id,
        (heure at time zone 'America/Toronto')::date as jour,
        sum(production) as vehicules_comptes

    from {{ source('argent', 'production_horaire') }}
    group by 1, 2

),

assembles as (

    select
        a.actif_id,
        p.cle_date_assemblage as jour,
        count(*) as vehicules_assembles

    from {{ ref('fct_production') }} as p
    inner join {{ ref('dim_actif') }} as a
        on a.cle_actif = p.cle_actif
```


© 2026 iAlexMG. Tous droits réservés.

```
    group by 1, 2

)

select
    coalesce(c.actif_id, a.actif_id) as actif_id,
    coalesce(c.jour, a.jour) as jour,
    c.vehicules_comptes,
    a.vehicules_assembles

from compteur as c
full outer join assembles as a
    on a.actif_id = c.actif_id and a.jour = c.jour

where c.jour is not null
  and coalesce(c.vehicules_comptes, 0) <> coalesce(a.vehicules_assembles, 0)
```

Trois points s'y jouent. La jointure est externe complète, comme toute réconciliation depuis le Cours 05. Le `where c.jour is not null` limite la comparaison à la fenêtre de l'historien, sans quoi les trois années que le compteur n'a pas vues échoueraient toutes. Et `heure at time zone 'America/Toronto'` convertit un instant absolu en heure locale avant d'en prendre la date : c'est la seule façon d'obtenir le jour de production qu'un contremaître reconnaîtrait.

```
dbt build --exclude snp_concessionnaire
```

Attendu : `PASS=103 WARN=0 ERROR=0` sur 104 nœuds, en quatre secondes. Le projet compte 27 modèles et 78 tests de données, dont 55 arrivés avec ce cours.

Puis, dans DuckDB, regardez ce que le test garantit :

```
with compteur as (
    select (heure at time zone 'America/Toronto')::date as jour, sum(production) as comptes
    from argent.production_horaire group by 1
)
select count(*) as jours, sum(comptes) as total_compteur,
       (select count(*) from marts.fct_production
         where cle_date_assemblage between date '2025-10-01' and date '2025-12-31') as total_erp
from compteur
where jour between date '2025-10-01' and date '2025-12-31';
```

Attendu : 92 jours, 1 833 véhicules comptés par les convoyeurs, 1 833 enregistrés par l'ERP. Deux systèmes qui ne se parlent pas, un même nombre.

Étape 19 — L'incident, d'un bout à l'autre

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_06_AE.md`, section 12.

Le 28 octobre 2025, le compresseur d'air 2 a déclenché. Quatre requêtes suffisent à lire toute l'histoire, et chacune interroge un fait différent.

La dérive, dans l'instantané :

```
select f.cle_date as jour,
       round(sum(f.part_en_marche), 1) as heures_de_marche,
       round(avg(f.vibration_moyenne) filter (where f.part_en_marche > 0.5), 2) as vibration_en_marche
from marts.fct_disponibilite_actifs f
inner join marts.dim_actif a on a.cle_actif = f.cle_actif
where a.actif_id = 'UTL-CMP2'
  and f.cle_date between date '2025-10-20' and date '2025-10-31'
group by 1 order by 1;
```

Jour	Heures de marche	Vibration en marche
2025-10-20	9,5	3,26
2025-10-22	9,5	4,48
2025-10-24	9,5	5,70
2025-10-27	9,5	7,68
2025-10-28	4,7	8,47
2025-10-30	1,9	2,42
2025-10-31	9,5	2,48

La dérive est nette, et le 28 se lit en une ligne : la machine s'arrête à la moitié de la journée. Le lendemain, c'est le compresseur d'appoint qui tourne, 9,5 heures, et la vibration du 2 est retombée à 2,4 après la réparation.

La trace humaine, dans la GMAO. Elle n'est pas dans l'étoile, et c'est voulu, aucun modèle ne la lit ; on la regarde directement dans la `RAW`, comme on ouvrirait le registre d'atelier :

```
select no_bt, ouvert_le, ferme_le, description
from raw.usine_bons_travail
where actif_id = 'UTL-CMP2' and ouvert_le like '%/10/2025%';
```

Deux bons : `BT-2025-00739`, ouvert le 22 octobre pour un bruit anormal, refermé en cinquante-deux minutes sans rien constater ; `BT-2025-00742`, ouvert le 28 à 10 h 50, soupape d'admission défaillante et roulement usé. Le premier est ce que la donnée avait déjà dit six jours plus tôt.

© 2026 iAlexMG. Tous droits réservés.

Les véhicules exposés, dans le fait de production :

```
select count(*) as motoneiges,
       min(numero_serie) as premiere, max(numero_serie) as derniere
from marts.fct_production p
inner join marts.dim_actif a on a.cle_actif = p.cle_actif
where a.actif_id = 'LMN'
  and p.cle_date_assemblage between date '2025-10-15' and date '2025-10-28';
```

Attendu : 240 motoneiges, de `MN26001238` à `MN26001477`, assemblées pendant que la pression du réseau d'air baissait.

Leur suite, dans les deux faits de l'après-vente :

```
with exposees as (
    select p.cle_vehicule
    from marts.fct_production p
    inner join marts.dim_actif a on a.cle_actif = p.cle_actif
    where a.actif_id = 'LMN'
      and p.cle_date_assemblage between date '2025-10-15' and date '2025-10-28'
)
select
    (select count(*) from exposees) as assemblees,
    (select count(*) from exposees x join marts.fct_enregistrements e
        on e.cle_vehicule = x.cle_vehicule) as vendues,
    (select count(distinct r.cle_vehicule) from exposees x join marts.fct_reclamations r
        on r.cle_vehicule = x.cle_vehicule) as avec_reclamation;
```

Attendu : 240 assemblées, 201 vendues au détail, 5 avec une réclamation. Cinq sur 201 font 2,5 pour cent, contre 3,3 pour cent chez les autres motoneiges de l'année-modèle 2026 : il n'y a rien à conclure, et c'est la bonne réponse. Les défauts d'un serrage insuffisant mettent des mois à paraître, et l'entrepôt s'arrête au 31 décembre 2025.

Ce que vous avez produit n'est donc pas un verdict, c'est une cohorte identifiée par ses numéros de série, prête à être resurveillée quand les mois de 2026 arriveront au Cours 07. C'est un usage d'entrepôt aussi légitime que les autres.

Étape 20 — La matrice de bus, remplie

Théorie : `Theorie_06_AE.md`, sections 10 et 12.

Le Cours 02 avait dessiné une matrice sur papier. La voici telle que le projet la porte :

© 2026 iAlexMG. Tous droits réservés.

Dimension	Ventes	Enregistrements	Réclamations	Production	Disponibilité
`dim_date`	oui	oui	oui	oui	oui
`dim_concessionnaire`	oui	oui	oui	non	non
`dim_produit`	oui	oui	non	oui	non
`dim_vehicule`	non	oui	oui	oui	non
`dim_client`	non	oui	oui	non	non
`dim_actif`	non	non	non	oui	oui

Vérifiez-la plutôt que de la croire, en demandant à dbt ce qui dépend de chaque dimension :

```
dbt ls --select dim_vehicule+
dbt ls --select dim_actif+
```

La première rend 10 nœuds, la seconde 9, et la surprise est dans ce qu'on n'y voit pas : aucun fait. Ils recalculent tous leurs clés par hachage plutôt que de joindre les dimensions, choix du Cours 04, donc rien ne les relie dans le graphe. Ce qu'on y voit à la place, ce sont les tests de relation, un par couple d'un fait et d'une dimension.

Autrement dit, la matrice de bus n'existe dans le projet que par ses tests. Retirez-les et il ne reste qu'une convention de hachage, que rien ne vérifie et que le graphe ignore. C'est la meilleure raison de les écrire qu'on puisse donner.

Les cinq faits couvrent cinq processus de Boréal, et il en reste un, la télémétrie des véhicules connectés, qui arrive au Cours 07 avec son chargement incrémental.

Récapitulatif

L'état du projet à la fin du cours :

Ce qui est en place	Vérification
Le projet complet	`dbt build --exclude snp_concessionnaire` rend `PASS=103 WARN=0 ERROR=0`
Les modèles	27, dont 15 arrivés avec ce cours
Les tests de données	78, dont 55 arrivés avec ce cours

© 2026 iAlexMG. Tous droits réservés.

Ce qui est en place	Vérification
`dim_actif`	23 actifs, 6 zones et 17 équipements
`fct_disponibilite_actifs`	22 090 lignes, 10 actifs par 2 209 heures, sans trou
`fct_production`	27 775 véhicules, du 2022-01-17 au 2025-12-23
Le témoin de l'usine	1 833 véhicules comptés par les convoyeurs, 1 833 par l'ERP, 0 jour en écart
L'incident	lu dans quatre faits, de la vibration du compresseur à la réclamation du client

Le Cours 07 met l'étoile en régime permanent : le lot mensuel arrive, et il n'est plus question de tout reconstruire à chaque exécution. C'est là que la télémétrie entre, avec ses 158 146 messages et ses doublons.

Pensez à commiter :

```
git add models tests
git commit -m "Cours 06 : l'etoile de l'usine"
```

Dépannage

Le message ou le symptôme	Ce qu'il veut dire	La parade
`Conversion Error: Malformed JSON at byte 0` sur une jointure	un `->>` sans parenthèses avant une comparaison : DuckDB l'a lu dans le mauvais ordre	encadrer l'extraction, `(objet ->> 'sernr') = e.sernr`
Le compte d'étapes est le double de l'attendu	l'historique a été déplié avant la déduplication	dédoublonner d'abord, déplier ensuite, étape 5
`dbt run --select int_jalons_reclamations` ne construit rien	le modèle est `ephemeral`	c'est normal : il n'existe que dans le SQL du fait qui l'appelle
Le fait des réclamations a plus de lignes que de réclamations	la jointure vers l'enregistrement ou vers le concessionnaire en ramène deux	un véhicule n'a qu'un enregistrement ; pour le concessionnaire, vérifier l'intervalle semi-ouvert du Cours 05
`not_null` échoue sur `cle_date_paiement`	ce test n'a rien à faire là	un jalon vide est un dossier en cours : seul `relationships` s'y pose

© 2026 iAlexMG. Tous droits réservés.

Le message ou le symptôme	Ce qu'il veut dire	La parade
Une heure manque dans l'instantané, ou une heure est en double	la nuit du changement d'heure, lue en heure locale	l'heure est un instant absolu : convertir explicitement pour obtenir un jour local, étape 18
Le témoin du compteur échoue sur trois années entières	la comparaison prend des jours que l'historien n'a jamais vus	borner la comparaison à la fenêtre de l'historien, étape 18
`fct_disponibilite_actifs` porte 17 actifs	le filtre sur les actifs qui ont un état de marche est tombé	seuls dix actifs en ont un, étape 15
La somme des montants des pièces ne tombe pas sur le total	les montants de la source sont arrondis au cent, et 1 277 réclamations n'ont aucune pièce	garder la mesure de la source et en déduire le reste, étape 9

© 2026 iAlexMG. Tous droits réservés.